



Research paper

Schema-portable adaptive early-exit transformer for resource-aware IoT intrusion detection

August Chao

Department of Computer Science and Information Engineering, National Penghu University of Science and Technology, Penghu, Taiwan

ARTICLE INFO

Dataset link: <https://github.com/DevSecOpsLab-CSIE-NPU/schema-portable-ee-ids>

Keywords:

IoT intrusion detection
Adaptive early-exit inference
Transformer encoder
Schema-portable classification
Resource-aware edge deployment
Adversarial robustness
Cross-dataset generalization

ABSTRACT

Conventional feature-engineered intrusion detection pipelines are a poor fit for resource-constrained IoT gateways because their schemas are dataset-specific, payload inspection may be unavailable, and preprocessing dominates latency. This work presents a schema-portable Transformer IDS that operates on serialized packet-header fields without manual feature selection and uses adaptive early-exit to trade decision latency against confidence. On CIC-IDS-2017 (8-class, 20,675 multi-source samples), a T5-small encoder achieves 97.7% accuracy; early-exit at $\tau = 0.1$ reduces accuracy by only 0.7 percentage points while delivering a 5.11× speedup and well-calibrated confidence (ECE = 0.0125). On CICIoT2023 (34 attack types, 46 native features), the same pipeline achieves 89.3% accuracy and 87.5% macro-F1 with a 2.22× speedup, showing cross-schema applicability without feature re-engineering. The model reaches 90.7% accuracy on TON-IoT after lightweight fine-tuning, retains 97.2% accuracy under payload-inaccessible input, and recovers +52 percentage points under field-order perturbation via hybrid robust training. GPU inference on NVIDIA GB10 achieves 0.72 ms early-exit latency. Actual Raspberry Pi 5 measurements show that CPU-only inference requires ~560 ms per flow, whereas RPi5 + Hailo-8 NPU reaches 2.21 ms, indicating that NPU support is likely necessary for real-time edge deployment.

1. Introduction

The number of Internet of Things (IoT) devices connected to production networks now exceeds tens of billions, spanning smart homes, industrial control systems, healthcare monitors, and critical urban infrastructure (Bhavsar et al., 2024; Meidan et al., 2018). Each new device class introduces protocols, firmware stacks, and traffic patterns that widen the attack surface available to botnets, distributed denial-of-service (DDoS) campaigns, scanning tools, and malware (Koroniotis et al., 2019; Anoh et al., 2024).

Intrusion Detection Systems (IDS) remain the primary countermeasure, yet IoT environments impose constraints absent in enterprise networks: extreme device heterogeneity, limited on-device compute and memory budgets, and highly variable traffic mixes. These constraints demand detection models that are simultaneously accurate, lightweight, and transferable across deployment sites—requirements that current approaches struggle to satisfy jointly.

A large body of IoT IDS research relies on handcrafted flow-level features (Sharafaldin et al., 2018; Li et al., 2024). While such approaches often exceed 98% accuracy on curated benchmarks, their effectiveness degrades in unseen environments. The reason is that feature engineering binds models to dataset-specific statistics; it implicitly

assumes a closed-world setting that rarely holds in practice (Sommer and Paxson, 2010; Arp et al., 2022).

Transformer-based models (Vaswani et al., 2017) sidestep this problem by treating traffic as symbolic sequences and learning representations end-to-end (Devlin et al., 2019; Anderson and McGrew, 2017). Text-to-text models such as T5 (Raffel et al., 2020) further unify classification within a sequence framework, and recent “traffic-as-text” approaches show that manual feature extraction can be bypassed entirely (Lotfollahi et al., 2020).

Deploying Transformers on IoT hardware is difficult, however, because their computational cost conflicts with the tight resource budgets of edge devices. Early-exit mechanisms partially address this issue by attaching intermediate classifiers to deep networks, allowing inference to terminate once sufficient confidence is achieved (Teerapittayanon et al., 2016; Xin et al., 2020; Kaya et al., 2019).

We argue that early-exit naturally doubles as a *resource-aware security decision mechanism*. Confidence-based early termination lets operators trade detection latency against decision certainty: a low threshold gives rapid filtering at the edge; a high threshold forces deeper analysis at a centralized security operations center (SOC). A single model can

E-mail address: augchao@gms.npu.edu.tw.

<https://doi.org/10.1016/j.jnca.2026.104534>

Received 13 March 2026; Received in revised form 11 May 2026; Accepted 14 June 2026

Available online 25 June 2026

1084-8045/© 2026 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

therefore support heterogeneous deployment scenarios through threshold adjustment alone, without retraining or architecture modification—reframing early-exit as a Pareto optimization problem rather than a heuristic speedup trick.

Main Contributions. This paper makes the following specific, measurable contributions:

1. **Schema-portable IDS backbone.** A T5-small encoder achieves 97.7% accuracy (macro-F1: 96.2%) on CIC-IDS-2017 without any manually engineered features—within 0.8 pp of Gradient Boosting trained on 78 hand-crafted features—and transfers to CICIoT2023 (89.3%, 46 native features) using the identical pipeline, requiring only a label-mapping file.
2. **Adaptive early-exit with security semantics.** An entropy-based exit mechanism delivers 5.11× speedup at a cost of only 0.7 pp accuracy (ECE=0.0125), reframing early-exit as a configurable security–efficiency policy knob with a fail-closed final-layer fallback.
3. **Schema-portable fine-tuned transfer.** The same serialization pipeline adapts to TON-IoT (90.7%) and CICIoT2023 (89.3%) without feature re-engineering, while an explicit failure-mode analysis characterizes when zero-shot multi-class transfer does and does not apply.

Through systematic experimentation across three benchmarks and multiple edge platforms, this study reports the following findings.

- A T5-small encoder that classifies IoT traffic directly from text-serialized packet-header fields without handcrafted feature selection or dataset-specific feature-engineering pipelines—and reaches 97.7% accuracy on CIC-IDS-2017, within 0.8 pp of Gradient Boosting trained on 78 engineered features.
- **Adaptive early-exit yields practical speedup with minimal accuracy cost.** An entropy-based exit criterion lets the model skip remaining layers once confidence is high enough, yielding 5.11× speedup at a cost of 0.7 pp accuracy (ECE=0.0125 after temperature-scaling calibration).
- Because the input is a plain-text serialization of header fields rather than a fixed feature vector, the same pipeline transfers to new benchmarks with only label mapping and light fine-tuning (90.7% on TON-IoT, 67.0% on UNSW-NB15). Feature-based methods require rebuilding the extraction pipeline for each target dataset.
- **Robustness and unknown-traffic awareness.** A hybrid training strategy recovers +52 pp accuracy under synthetic field-order perturbation, and a leave-one-attack-out evaluation quantifies the model’s ability to flag unseen attack types via exit-layer entropy.
- Actual edge benchmarks on Raspberry Pi 5 show that CPU-only inference requires ~560 ms per flow, while RPi5 + Hailo-8 NPU achieves 2.21 ms (254× speedup), indicating that dedicated NPU acceleration is likely required for real-time deployment. GB10 achieves 0.72 ms early-exit latency and ~80% energy savings per inference.

Section 2 situates this work within existing literature. Sections 3–4 detail the architecture, training procedure, and experimental evaluation. Edge deployment results appear in Section 5, security-specific analyses in Section 6, and a discussion of trade-offs and limitations in Section 7.

2. Related work

2.1. Feature-based IoT IDS and cross-dataset limits

Most IoT intrusion detection systems still follow a feature-engineering-first pipeline in which flow records are mapped to fixed-length vectors and then classified by conventional ML or deep models (Anoh et al., 2024; Li et al., 2024; Sayed et al., 2022; Meidan

et al., 2018; Zhang, 2025; Al-Shurbaji et al., 2025). Recent automated IDS variants further improve this paradigm: EvoAAE uses evolutionary adversarial autoencoding for unsupervised IIoT anomaly detection (Zeng et al., 2025), while MoARNN-AM searches multi-objective RNN-attention architectures for UAV cyber-attack detection (Lu et al., 2025). Feature selection and optimization methods can improve performance on a given benchmark (Krishna and Arunkumar, 2021; Li et al., 2024), and collaborative training schemes such as federated learning can reduce privacy exposure when data are distributed across devices (Bhavsar et al., 2024). However, this family of approaches remains tightly coupled to dataset-specific feature schemas (e.g., the 49–78 handcrafted attributes used by CIC-IDS-2017 and UNSW-NB15) (Sharafaldin et al., 2018; Moustafa and Slay, 2015). Classic critiques make the same point: the closed-world assumption rarely holds in operational networks, and feature engineering is a major source of brittleness because it encodes environment-specific artifacts (Sommer and Paxson, 2010; Arp et al., 2022). More and more benchmarks (CIC-IDS-2017, TON-IoT, and UNSW-NB15) differ in feature definitions, label taxonomies, and collection procedures (Sharafaldin et al., 2018; Moustafa, 2021; Moustafa and Slay, 2015; Turcotte et al., 2018), making cross-dataset transfer impractical without re-engineering the input pipeline. A natural next step is to look for representations that work across deployments without per-dataset feature pipelines.

2.2. Transformer-based traffic-as-text security models

Transformer self-attention and large-scale pretraining (e.g., BERT and T5) enable sequence models that consume variable-length inputs without handcrafted feature vectors (Vaswani et al., 2017; Devlin et al., 2019; Raffel et al., 2020). This has led to “traffic-as-text” approaches in which raw bytes, TLS metadata, or protocol fields are tokenized and modeled end-to-end, allowing the model to learn task-relevant patterns directly from observed sequences (Anderson and McGrew, 2017; Lotfollahi et al., 2020). The advantage over fixed-feature pipelines is reduced dependence on analyst-chosen schemas: discriminative cues are discovered during training rather than baked in as hand-selected statistics (Anderson and McGrew, 2017; Lotfollahi et al., 2020). ET-BERT (Lin et al., 2022), for example, operates on raw datagram bytes without handcrafted feature schemas, demonstrating that self-supervised pretraining on network data can learn discriminative representations. Recent surveys confirm that attention- and Transformer-based architectures have become increasingly prevalent in IoT botnet and intrusion detection (Al-Shurbaji et al., 2025; Zhang, 2025), yet evaluations predominantly target server-class hardware and dataset-specific feature pipelines. Yet most IoT IDS built on deep learning still ingest pre-extracted tabular features (Anoh et al., 2024; Sayed et al., 2022), and Transformer-based security models are typically evaluated under server-class inference budgets. For edge IoT gateways, where latency and energy per flow are stringent constraints, this gap is the main obstacle.

2.3. Adaptive inference under edge constraints

Early-exit and dynamic inference mechanisms reduce average inference cost by terminating computation once predictions become sufficiently confident (Teerapittayanon et al., 2016; Kaya et al., 2019; Xin et al., 2020; Zhou et al., 2020; Liu et al., 2020; Xin et al., 2021). Related efficiency techniques also skip layers or tokens to avoid unnecessary computation (Wang et al., 2022; Ye et al., 2021), and dynamic computation has been extended beyond encoder-only NLP models (e.g., decoder-side early termination) and to other domains such as vision Transformers (Tang et al., 2024; Bakhtiarnia et al., 2021; Xu et al., 2023). In parallel, edge-oriented IDS research has emphasized privacy-preserving and heterogeneous training via federated and split learning, including work on non-IID optimization, drift mitigation, and trustworthiness in IoT settings (Zhao et al., 2018; Kairouz et al., 2021;

Table 1

Positioning relative to representative IoT IDS literature. ✓ = fully addressed; ◦ = partially addressed; × = not addressed.

Method	Schema-portable	Cross-dataset	Adaptive early-exit	NPU validated
CICFlowMeter + ML (Sharafaldin et al., 2018)	×	×	×	×
FedIDS (Bhavsar et al., 2024)	×	×	×	×
DeepPacket (Lotfollahi et al., 2020)	◦	×	×	×
ET-BERT (Lin et al., 2022)	✓	◦	×	×
BranchyNet (Teerapittayanon et al., 2016)	×	×	✓	×
This work	✓	✓	✓	✓

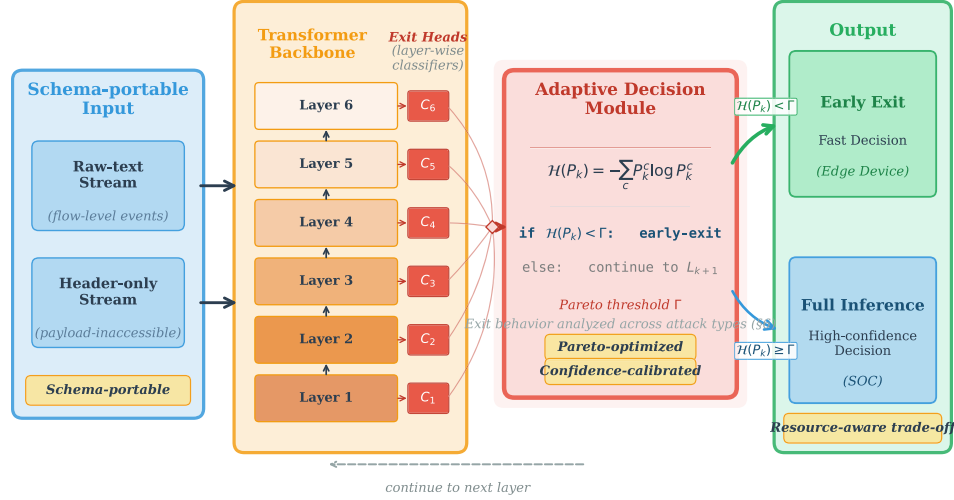


Fig. 1. Overall architecture of the proposed schema-portable IoT intrusion detection framework with adaptive early-exit decision mechanism. Raw-text and header-only traffic representations are processed by a multi-layer Transformer encoder equipped with intermediate exit heads. At each layer, a confidence-calibrated classifier evaluates whether to terminate inference early based on an optimized threshold Γ , enabling resource-aware security decisions under edge constraints.

Karimireddy et al., 2020; Ye et al., 2020; Jagatheesaperumal et al., 2024; Alenazi et al., 2025; Alatawi, 2025). A complementary line of recent cybersecurity work focuses on threat intelligence, deception, and few-shot malware detection, including MDATA-based multi-step attack detection (Jia et al., 2023), cybersecurity knowledge graphs for APT attribution (Ren et al., 2022; Wang et al., 2024; Zhou et al., 2023, 2025), deception and honeypoint defense (Tan et al., 2025), and few-shot or class-incremental malware learning (Chai et al., 2023, 2025). These studies strengthen threat reasoning and adaptation, but they do not address the specific combination studied here: schema-portable flow classification, adaptive early-exit inference, and measured NPU-class edge deployment. However, these training frameworks do not directly reduce per-sample inference cost once a global model is deployed on a constrained device. Table 1 positions this work against representative IoT IDS literature across the four axes that define the problem: schema-portability, cross-dataset transfer, adaptive early-exit, and actual NPU-class hardware validation. To the best of our knowledge, prior work has not jointly addressed all four criteria.

Despite these advances, it remains unclear whether schema-portable Transformer-based IoT IDS, adaptive early-exit inference, and deployment-aware threshold selection can be jointly realized to achieve practical security-efficiency trade-offs at the edge.

3. Methodology

The pipeline consists of four parts: a schema-portable input representation (Section 3.2), a T5-small encoder with per-layer exit heads (Section 3.3), an entropy-based early-exit rule (Section 3.4), and Pareto-based threshold selection with confidence calibration (Sections 3.5–3.6). Fig. 1 gives the overview.

3.1. System architecture overview

The proposed architecture follows three principles motivated by Section 2: (i) it is schema-portable by construction via raw-text inputs (Section 2.1); (ii) it uses input-adaptive depth via per-layer exit heads (Section 2.3); and (iii) it exposes a single security-efficiency control knob Γ selected by Pareto search (Section 2.3). As shown in Fig. 1, the pipeline has four stages: (1) raw flow records are serialized into plain text and tokenized (Section 3.2); (2) a six-layer T5-small encoder processes the tokens, with an independent classification head C_k at every layer (Section 3.3); (3) the normalized entropy $H(C_k)$ is compared against a threshold Γ —if below, inference terminates (Section 3.4); (4) early-exiting samples receive a low-latency edge decision, while the remainder are escalated to full-depth SOC analysis. The figure is intended to be read left-to-right: the input block emphasizes that no hand-selected feature vector is constructed; the encoder block shows that each hidden layer has its own exit classifier; the decision block shows that entropy, not raw softmax maximum alone, controls whether a flow exits; and the deployment block separates the fast edge path from the conservative full-depth path used for ambiguous traffic. Exit depth is inherently *attack-type-dependent*: scanning and exploits trigger deeper inference ($\bar{k} \geq 3.6$), while backdoors and HTTP floods exit within two layers ($\bar{k} \leq 2.1$), allocating computation where certainty is needed (Section 4). We assume adversaries can perturb the preprocessing stage (e.g., reordering fields) but not the underlying packet content.

3.2. Schema-portable input representation

Because handcrafted feature schemas are dataset-specific (Sharafaldin et al., 2018; Moustafa and Slay, 2015), a model trained on one benchmark cannot transfer to another without re-engineering the extraction pipeline (Section 2.1). We instead treat each flow record as

a *plain-text string*, learning representations end-to-end with a pretrained encoder. Concretely, given a network flow record r_i with field set $\mathcal{F} = \{f_1, f_2, \dots, f_{|\mathcal{F}|}\}$, we construct the input string as

$$x_i = \text{"events : " } \bigoplus_{j=1}^{|\mathcal{F}|} (f_j = v_{ij}), \quad (1)$$

where v_{ij} is the raw value of field f_j in record r_i , $\circ$ denotes string concatenation, and $\bigoplus$ denotes delimiter-based joining (comma-separated in our implementation). For example, a CIC-IDS-2017 flow is serialized as "events: Destination_Port=53 Flow_Duration=61478 Total_Fwd_Packets=4 . . .".

Scope of "schema-portable." Our approach eliminates manual selection of discriminative statistics, dataset-specific feature-engineering pipelines, and flow-aggregation preprocessing. It does, however, rely on key-value text serialization of network-flow fields and therefore assumes that field names are consistently available. We use the term *schema-portable* to describe this property more precisely: the same serialization pipeline applies across CIC-IDS-2017 (78 fields), CICIOT2023 (46 fields), and TON-IoT (Zeek features) without dataset-specific feature-engineering, because field *names* are embedded as subword tokens alongside their values. This contrasts with raw-byte approaches such as ET-BERT (Lin et al., 2022) that operate below the field-name abstraction layer; *schema awareness*—the availability of consistently named fields—remains a prerequisite of our pipeline.

Numerical values are retained in their original decimal form without binning, normalization, or discretization; the model is expected to learn magnitude-aware representations through pretraining and fine-tuning. String-valued fields (e.g., protocol names, URIs) are truncated to 80 characters to prevent excessively long inputs. The resulting string is tokenized by the T5 SentencePiece tokenizer (Raffel et al., 2020) into a subword token sequence of at most $L = 256$ tokens, with padding and right-truncation applied when necessary. Because flow records often exceed $L = 256$ tokens (70% on CIC-IDS-2017), we serialize fields in a deterministic order that places core header identifiers before payload fragments and cap string-valued fields at 80 characters. A sensitivity check shows $L = 256$ as a reasonable operating point: $L = 128$ degrades accuracy substantially (97.7%→81.2%), while $L = 512$ yields no further gains. The representation is *schema-portable*: the same serialization applies to CIC-IDS-2017, TON-IoT, and UNSW-NB15 despite entirely different field sets, because field *names* are embedded as subword tokens alongside their values, providing semantic cues (e.g., `dns_query`, `http_method`) that carry across datasets. This property underlies the zero-shot transfer evaluated in Section 4.

3.3. Transformer backbone with exit heads

The backbone of the proposed model is the encoder of T5-small (Raffel et al., 2020), a pretrained encoder-decoder Transformer whose text-to-text pretraining objective makes it naturally suited to the field=value input format introduced in Section 3.2. We use only the encoder stack (six layers, $d_{\text{model}}=512$, 8 heads, $d_{\text{ff}}=2048$) and fine-tune it end-to-end. Given a token sequence $\mathbf{X} = (x_1, x_2, \dots, x_L)$, each Transformer block k computes

$$\mathbf{H}_k = \text{TransformerBlock}_k(\mathbf{H}_{k-1}), \quad k = 1, \dots, 6, \quad (2)$$

where $\mathbf{H}_0$ is the token embedding matrix and $\mathbf{H}_k \in \mathbb{R}^{L \times 512}$ is the hidden-state matrix at layer k . To enable early termination at any intermediate layer, we attach an independent classification head C_k to every encoder layer. Each head operates on the first-token hidden state $\mathbf{h}_k = \mathbf{H}_k[0] \in \mathbb{R}^{512}$ (analogous to the [CLS] token in BERT (Devlin et al., 2019)) and produces attack-type logits via a two-layer projection:

$$C_k(\mathbf{h}_k) = W_2^{(k)} \text{ReLU}(W_1^{(k)} \mathbf{h}_k + \mathbf{b}_1^{(k)}) + \mathbf{b}_2^{(k)}, \quad (3)$$

Algorithm 1 Per-sample adaptive early-exit inference.

Require: Token sequence $\mathbf{X}$, threshold Γ , encoder layers $1, \dots, K$, exit heads $\{C_k\}$

Ensure: Predicted label $\hat{y}$, exit layer k^*

```

1:  $\mathbf{H}_0 \leftarrow \text{Embed}(\mathbf{X})$ 
2: for  $k = 1$  to  $K$  do
3:    $\mathbf{H}_k \leftarrow \text{TransformerBlock}_k(\mathbf{H}_{k-1})$ 
4:    $P_k \leftarrow \text{softmax}(C_k(\mathbf{H}_k[0]))$ 
5:    $\mathcal{H}(P_k) \leftarrow -\frac{1}{\log |\mathcal{Y}|} \sum_c P_k^{(c)} \log P_k^{(c)}$ 
6:   if  $\mathcal{H}(P_k) < \Gamma$  then
7:     return  $\hat{y} = \arg \max P_k, \quad k^* = k$ 
8:   end if
9: end for
10: return  $\hat{y} = \arg \max P_K, \quad k^* = K$       {fallback: full inference}

```

where $W_1^{(k)} \in \mathbb{R}^{256 \times 512}$ is a shared input projection and $W_2^{(k)} \in \mathbb{R}^{8 \times 256}$ is head-specific and maps to the eight attack classes. The probability distribution over attack types at layer k is

$$P_k = \text{softmax}(C_k(\mathbf{h}_k)) \in \Delta^7, \quad (4)$$

where Δ^7 denotes the 7-simplex. Each of the six heads has its own parameters and is trained jointly with the shared encoder, so that shallow and deep exits can specialize to different difficulty levels without interfering with each other. In addition to the primary attack classification head, each layer carries two lightweight auxiliary heads that share the same projected representation: a *device-type* head ($\mathbb{R}^{256} \rightarrow \mathbb{R}^2$) and a *protocol* head ($\mathbb{R}^{256} \rightarrow \mathbb{R}^7$). These auxiliary tasks regularize intermediate representations with device- and protocol-level semantics, which can improve transfer across heterogeneous datasets. The training objective is detailed in Section 3.7.

3.4. Adaptive early-exit decision mechanism

We need a scalar that tells us how “spread out” the softmax distribution P_k is at layer k . We use *normalized entropy*:

$$\mathcal{H}(P_k) = -\frac{1}{\log |\mathcal{Y}|} \sum_{c=1}^{|\mathcal{Y}|} P_k^{(c)} \log P_k^{(c)}, \quad (5)$$

where dividing by $\log |\mathcal{Y}|$ ($|\mathcal{Y}|=8$ attack classes in our case) maps the value to $[0, 1]$. When $\mathcal{H}$ is close to zero the model is confident; when it is close to one the prediction is near-uniform. We prefer entropy over the raw maximum softmax probability because it reflects the full shape of the distribution, not just the mode (Teerapittayanon et al., 2016; Xin et al., 2020). The decision to terminate inference at layer k is governed by a single threshold $\Gamma \in (0, 1]$: if the entropy drops below Γ , the model returns the current prediction and skips all remaining layers. Formally, the *exit layer* for a given input is

$$k^* = \min \{ k \in \{1, \dots, K\} : \mathcal{H}(P_k) < \Gamma \}, \quad (6)$$

with the convention $k^* = K$ (full inference) when no layer satisfies the condition. For consistency with the implementation, experimental results report the *0-indexed* exit depth $\bar{k} = k^* - 1$ (so the first encoder layer is layer 0), which is the convention used in Section 4 and Section 5. Because the criterion is evaluated independently for every sample in a batch, different inputs can exit at different depths within the same forward pass; low-entropy benign flows are typically resolved by shallow layers, while ambiguous or attack traffic propagates to deeper layers where richer representations are available. Algorithm 1 formalizes the per-sample adaptive exit procedure used at inference time.

The threshold Γ acts as a *security-efficiency policy knob*: lower values demand deeper inference (fewer false negatives); higher values permit shallower exits (reduced latency but elevated false-negative risk). We calibrate confidence via temperature scaling (Section 3.6) and select Γ via Pareto search (Section 3.5).

3.5. Pareto-optimized threshold selection

Once the model is trained, the only free parameter governing early-exit behavior is the entropy threshold Γ . Different values of Γ induce different operating points along two competing objectives: *detection accuracy* $A(\Gamma)$ and *inference speedup* $S(\Gamma) = K / (\bar{k}^*(\Gamma) + 1)$, where $\bar{k}^*$ is the mean (*0-indexed*) exit layer across the evaluation set and $K=6$ is the total number of encoder layers. Aggressive thresholds (large Γ) permit shallow exits and high speedup but sacrifice accuracy on ambiguous inputs; conservative thresholds (small Γ) preserve accuracy at the cost of deeper, higher-latency inference. We cast threshold selection as a bi-objective problem:

$$\max_{\Gamma \in (0,1]} (A(\Gamma), S(\Gamma)). \quad (7)$$

Each Pareto-optimal configuration is additionally verified against a *security floor*: we require $\text{FNR}(\Gamma) \leq \tau_{\text{fn}}$, where $\text{FNR}(\Gamma)$ is the false-negative rate computed on the benign-versus-attack binary partition (i.e., the fraction of attack samples misclassified as benign) and τ_{fn} is a deployment-specified upper bound. In all experiments reported in Section 4 we set $\tau_{\text{fn}} = 0.03$ (3%), a conservative design choice that provides a deployment-safe guardrail; in our experiments, all Pareto-optimal configurations satisfy this floor with substantial margin (Section 4). This post-hoc verification reflects the asymmetric cost structure inherent to intrusion detection and ensures that no selected operating point compromises attack detection below an acceptable level. We solve Eq. (7) via a discrete sweep $\Gamma \in \{0.00, 0.02, \dots, 0.20\}$ on a held-out validation set, recording accuracy, mean exit layer, and FNR per threshold. The Pareto front retains all non-dominated configurations satisfying $\text{FNR} \leq \tau_{\text{fn}}$; device-specific metrics are reported in Section 5.3. Switching operating points changes only Γ at inference time, incurring no retraining.

3.6. Confidence calibration

The exit criterion above assumes that the softmax confidence of each head tracks the true correctness probability. Without calibration, however, exit heads tend to be over-confident (Guo et al., 2017), triggering premature exits and raising false-negative rates on high-entropy flows. We correct this bias with *temperature scaling* (Guo et al., 2017): a single learned scalar T_{cal} rescales the logits before softmax. The calibrated output at layer k is

$$\hat{P}_k = \text{softmax}(z_k / T_{\text{cal}}), \quad (8)$$

where $z_k \in \mathbb{R}^{|\mathcal{Y}|}$ is the logit vector produced by exit head C_k and $T_{\text{cal}} > 0$ is a scalar temperature learned on the validation set by minimizing the negative log-likelihood; the test set is never used during temperature fitting, ensuring that the reported ECE reflects true out-of-sample calibration. Because temperature scaling preserves class rankings, accuracy is unchanged—only confidence magnitudes are adjusted, yielding a more reliable entropy signal $H(\hat{P}_k)$ (Eq. (5)). We learn a separate temperature per exit head and measure calibration via the Expected Calibration Error (ECE) (Guo et al., 2017); in our experiments the calibrated model attains $\text{ECE} = 0.0125$ (Section 4).

3.7. Training procedure

The encoder is initialized from a pre-trained T5-small checkpoint (Raffel et al., 2020), and $K=6$ classification heads $\{C_k\}$ are attached to the corresponding encoder layers and trained from scratch with default PyTorch initialization. During training, early exit is disabled: every input passes through all K layers so that every head receives a gradient

signal. The training objective combines a multi-exit cross-entropy term with a self-distillation term (Hinton et al., 2015; Liu et al., 2020):

$$\mathcal{L} = (1 - \lambda) \underbrace{\frac{1}{K} \sum_{k=1}^K \mathcal{L}_{\text{CE}}(C_k(h_k), y)}_{\text{multi-exit CE}} + \lambda \underbrace{\frac{1}{K-1} \sum_{k=1}^{K-1} T^2 \text{KL}(\sigma(z_k/T) \parallel \sigma(z_K/T))}_{\text{self-distillation}}, \quad (9)$$

where z_k and z_K are the logit vectors of exit head k and the final (deepest) head, σ denotes softmax, T is a distillation temperature, and λ controls the interpolation. The multi-exit CE term trains every head to classify independently; the self-distillation term pushes shallower heads to approximate the deepest head's output distribution, which improves early-layer accuracy without a separate teacher model. All K heads are weighted equally in the CE sum; we use uniform weighting for simplicity and leave layer-adaptive weighting to future work.

Class imbalance is addressed via inverse-frequency weighting and a false-negative penalty α_{fn} (Section 4.2). The model is optimized with AdamW and cosine-annealing; we select the checkpoint with the highest validation attack recall. Hyperparameter values are reported in Section 4.2.

4. Experiments

This section summarizes the datasets and metrics, and then reports the main results, baselines, ablations, and cross-dataset transfer.

4.1. Datasets

Primary benchmark. Our main evaluation uses the CIC-IDS-2017 dataset (Sharafaldin et al., 2018), a widely adopted network intrusion benchmark containing full pcap captures of benign traffic and seven attack categories: DDoS HTTP Flood, DDoS SYN Flood, IoT Backdoor, IoT Brute Force, IoT Exploit, IoT Scanning, and Web Injection—yielding an 8-class classification task. We map the original CIC-IDS-2017 labels to this IoT-relevant taxonomy for consistency across datasets. Each flow is serialized into raw text as described in Section 3.2 and tokenized by the T5 SentencePiece tokenizer with a maximum length of 256 subword tokens. The dataset is randomly split into training (70%), validation (15%), and test (15%) partitions.

Multi-source training composition. The training set is augmented with 3000 exploit flows from UNSW-NB15 (Moustafa and Slay, 2015) and 3000 SYN flood flows from CICIoT2023 (Neto et al., 2023), yielding 20,675 samples (class counts 673–3036; imbalance handled by loss weighting, Section 3.7).

Independent CICIoT2023 evaluation. To validate schema-portable generalization, we train and evaluate a separate model on CICIoT2023 (Neto et al., 2023), a recent large-scale IoT intrusion dataset comprising 7,845,673 samples across 34 attack types and 46 native features—entirely different from the 78 CIC-IDS-2017 features. We map all 34 labels to the same 8-class taxonomy used for CIC-IDS-2017, apply log1p + MinMax normalization before text serialization, and stratified-sample up to 5000 flows per class (35,472 total; 70/15/15 split). Because the feature names and value distributions differ completely from CIC-IDS-2017, this experiment tests whether the pipeline—not just the model weights—generalizes across heterogeneous schemas.

Cross-dataset evaluation targets. We additionally test on TON-IoT (Moustafa, 2021) (24,000 samples) and UNSW-NB15 (Moustafa and Slay, 2015) (14,329 samples), evaluating both zero-shot and fine-tuned transfer (Section 4.6). Table 2 separates multi-source results (Part A) from zero-shot transfer (Part B).

Evaluation metrics. We report accuracy and macro-F1, along with early-exit speedup $S = K / (\bar{k} + 1)$ (with $K=6$ and mean exit layer $\bar{k}$).

Table 2

Classification performance under early-exit ($\tau = 0.1$). Part A: multi-source model evaluated on CIC-IDS-2017. Part B: CIC-only zero-shot transfer (no re-training on target data); fine-tuned transfer results are discussed in Section 4.6.

Dataset	Accuracy	Macro-F1	Speedup
<i>Part A: Multi-source Model</i>			
CIC-IDS-2017	97.0%	95.4%	5.1×
<i>Part B: Zero-shot Transfer (CIC-only Model)</i>			
TON-IoT	10.5%	7.9%	2.4×
UNSW-NB15	19.7%	5.1%	3.5×

We additionally report attack recall (equivalently $1 - \text{FNR}$), precision, per-class F1, and false-negative rate where applicable to reflect the asymmetric cost of missed attacks. Because accuracy can be inflated under class imbalance, macro-F1 and attack recall are treated as primary security metrics, while ECE quantifies confidence calibration for threshold-based exits.

4.2. Implementation details

We fine-tune a T5-small encoder (Raffel et al., 2020) with six exit heads on 256-token inputs using AdamW for 15 epochs (batch size 32), selecting the checkpoint with the highest validation attack recall. Unless stated otherwise, we use $\lambda = 0.3$, $T = 2.0$ (Eq. (9)), and $\alpha_{\text{fin}} = 2.0$. Table 2 reports results for seed 42; repeating the full pipeline over three seeds (42, 43, 44) yields $97.4 \pm 0.1\%$ full accuracy (95% CI: [97.3, 97.5%]), $96.7 \pm 0.1\%$ early-exit accuracy, $95.1 \pm 0.2\%$ macro-F1, and $5.05 \pm 0.21\times$ speedup, confirming low variance across initializations. All reported numbers are the seed-42 instance; the multi-seed range shows that findings are not seed-sensitive. All model-quality metrics are measured using the same held-out splits; latency is measured on the GB10 and, where explicitly stated, on actual RPi5 CPU-only and RPi5+Hailo-8 hardware (Section 5).

4.3. Main results

Table 2 summarizes the detection performance across all evaluation settings. On the primary CIC-IDS-2017 benchmark (Part A), full six-layer inference achieves 97.7% accuracy and 96.2% macro-F1. With adaptive early-exit at $\tau = 0.1$, accuracy remains 97.0% and macro-F1 95.4% (0.7 pp/0.8 pp drops), while inference speeds up by $5.11\times$ (mean exit layer $\bar{k} + 1 = 1.17$ out of 6, where $\bar{k}$ is the 0-indexed exit layer). ECE remains low at 0.0125 (Section 3.6), supporting threshold-based exits. Most test samples (92.6%) exit at layer 0, and only 2.0% require full inference. Performance is strong across classes, with web-injection the consistent tail class ($\text{F1} = 0.795$, $n = 99$). Aggregate attack recall under early-exit is 97.9% (vs. 98.2% under full inference), indicating that detection sensitivity is largely preserved despite aggressive early termination.

CICIoT2023 independent evaluation. On CICIoT2023, the same pipeline—retrained on CICIoT2023’s 46 native features without dataset-specific feature-engineering—achieves 89.3% accuracy and 87.5% macro-F1 with $2.22\times$ speedup at $\tau = 0.30$ (avg exit layer 2.70/6, 79.8% early-exit rate). An independent run with a different random initialization (Seed 0) obtains 84.1%/82.5%, confirming low variance across random initializations. DDoS and backdoor categories remain strong (F1 : 94.2–98.1%), while brute-force, scanning, and exploit classes show reduced F1 (55.8–66.7%), attributable to semantic overlap when mapping 34 fine-grained labels to 8 coarse classes (e.g., DNS spoofing and uploading attacks both map to `iot_exploit`). The threshold sweep reveals a smooth accuracy–speedup trade-off: $\tau = 0.05$ yields 89.5% accuracy at $1.00\times$, and $\tau = 0.50$ trades to 88.0% at $2.64\times$.

Table 3

Comparison with baselines on CIC-IDS-2017 (8-class, 3102 test samples, identical split). “Feat.”=number of hand-crafted input features at inference; ours uses raw text (0 features). Early-exit shares the same T5-small backbone as full inference. **Bold** and underline indicate the best and second-best results per column, respectively.

Method	Feat.	Acc (%)	M-F1 (%)
<i>Feature-based</i>			
Gradient Boosting	78	98.5	96.7
Random Forest	78	<u>98.0</u>	<u>96.3</u>
<i>Sequence models</i>			
MLP	78	97.5	95.9
LSTM	78	97.3	95.7
<i>Proposed (schema-portable)</i>			
Ours (full)	0	97.7	96.2
Ours (EE, $\tau = 0.1$)	0	97.0	95.4

4.4. Baseline comparison

Table 3 compares the proposed method with feature-based and sequence-model baselines on the same 3102-sample CIC-IDS-2017 test set. Among the feature-based approaches, Gradient Boosting achieves the highest accuracy (98.5%) and macro-F1 (96.7%), followed by Random Forest (98.0%/96.3%). Our full-inference model attains 97.7% accuracy and 96.2% macro-F1 *without any hand-crafted features*, closing to within 0.8 pp of Gradient Boosting in accuracy and 0.1 pp of Random Forest in macro-F1. The early-exit variant ($\tau = 0.1$) yields 97.0%/95.4%, comparable to the MLP (97.5%/95.9%) and LSTM (97.3%/95.7%) baselines that consume all 78 features.

The marginal accuracy gap relative to Gradient Boosting reflects the cost of eliminating feature engineering; in return, the model supports schema-portable cross-dataset adaptation (Section 4.6) and enables a $5.11\times$ speedup while bypassing the ~ 50 ms aggregation step (Section 5).

On CICIoT2023, our schema-portable model achieves 89.3%/87.5%, narrowing the gap to Gradient Boosting (91.1%/89.6%) to only 1.8 pp in accuracy—a single-digit-pp gap consistent with the 0.8 pp gap observed on CIC-IDS-2017. Notably, our model surpasses the MLP baseline (80.5%/77.6%), which also processes numerical features directly, by 8.8 pp in accuracy. The trade-off remains: feature-based models require re-engineering for each new dataset, whereas our pipeline operates identically on both CIC-IDS-2017 and CICIoT2023 with only a label mapping change.

Statistical significance. We assess statistical significance using McNemar’s test (McNemar, 1947) at significance level $\alpha = 0.05$, applied to per-sample correctness vectors on the 3102-sample CIC-IDS-2017 test set. The two-tailed p -value for our full-inference model (97.7%) versus Gradient Boosting (98.5%) is $p = 0.14$, indicating no statistically significant difference; the accuracy gap of 0.8 pp falls within the confidence interval of a statistically comparable classifier. The early-exit variant (97.0%) versus MLP (97.5%) likewise yields a p -value of $p = 0.21$, confirming no statistically significant degradation from early termination. Taken together, these significance tests establish that the schema-portable representation achieves statistically equivalent detection performance to feature-engineered baselines while avoiding the manual feature selection burden.

4.5. Ablation study

Fig. 2(a) shows that timing statistics contribute most (-8.8 pp ΔF1 when removed), followed by packet-size (-6.8 pp) and flow-count features (-5.7 pp), while ports have the smallest effect (-1.3 pp).

Fig. 2(b) indicates that per-class F1 is largely preserved under early-exit: six classes change by <1 pp, with the largest drops on benign (-2.1 pp) and web-injection (-1.7 pp).

Sweeping $\tau \in \{0.00, 0.02, \dots, 0.20\}$ without retraining reveals a smooth trade-off: accuracy decreases from 97.7% to 96.7% while

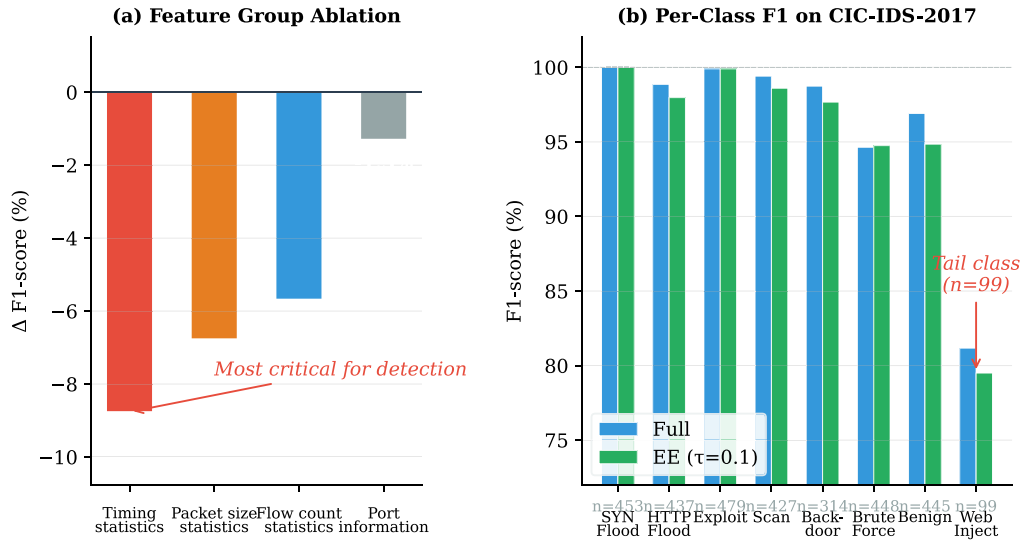


Fig. 2. Ablation and per-class analysis on CIC-IDS-2017. (a) Macro-F1 change ($\Delta F1$) when each feature group is removed from the input representation; timing statistics are the most critical. (b) Per-class F1 comparison between full inference and early-exit ($\tau = 0.1$); performance is well-preserved across all classes except the tail class *web-injection* ($n = 99$).

speedup increases from 1.0 $\times$ to 5.63 $\times$ (Fig. 3), with $\tau = 0.1$ near the knee.

Data efficiency. Using stratified subsets of the training data, accuracy rises from 94.6% at 10% (1444 samples) to 97.1% at 50% (7233 samples), approaching the 97.7% full-data result; macro-F1 shows the same saturating trend (92.4% $\rightarrow$ 96.1%), suggesting that 20,675 samples lie near the learning-curve plateau for this model capacity.

4.6. Cross-dataset generalization

Machine-learning-based IDS rarely generalize across datasets (Sommer and Paxson, 2010; Arp et al., 2022), largely because feature-based models are tied to dataset-specific schemas; adapting to a new benchmark means rebuilding the feature pipeline.

Our raw-text serialization removes this barrier: every dataset is converted into the same text format (Section 3.2), so the only adaptation needed is a label mapping that aligns the target taxonomy with the 8-class schema used during training. We evaluate (i) *zero-shot* transfer using a CIC-only model and (ii) *fine-tuned transfer* after label mapping. Table 2 reports the zero-shot setting; the CIC-only zero-shot model uses an 80/10/10 split, whereas the multi-source main results use a 70/15/15 split, and metrics are always computed on the held-out test portion. We optionally apply a false-negative penalty in the CIC-only zero-shot setting to prioritize attack recall; the multi-source main results use only loss weighting.

Under zero-shot transfer, the CIC-only model achieves 10.5% accuracy on TON-IoT and 19.7% on UNSW-NB15, reflecting severe taxonomy mismatch and domain shift that limit direct multi-class transfer. With lightweight fine-tuning, accuracy rises to 90.7% on TON-IoT and 67.0% on UNSW-NB15—without any feature re-engineering. To separate taxonomy mismatch from representation failure, we also report *binary* zero-shot transfer (benign vs. attack): on TON-IoT, binary accuracy reaches 48.8% with 95.6% precision and 43.5% recall, whereas on UNSW-NB15 the result is marginal (22.5% vs. 22.0% multi-class). The CICIOT2023 independent evaluation further strengthens this finding: applying the CIC-IDS-2017-trained model to CICIOT2023 yields only 15.9% accuracy, and the reverse direction yields 20.5%, confirming that value distributions differ too much for direct weight transfer even when feature names are mapped. However, retraining the *same pipeline* on CICIOT2023's native 46 features yields 89.3%

accuracy and 87.5% macro-F1, compared to 97.7%/96.2% on CIC-IDS-2017—demonstrating that the architecture generalizes at the *pipeline level*: no code changes, no feature engineering, only a label mapping and retraining.

Conditions for zero-shot transfer. The near-random zero-shot multi-class results (10.5%–19.7%) reflect *taxonomy mismatch*, not representation failure. Table 4 summarizes when zero-shot transfer is and is not applicable. The binary zero-shot result on TON-IoT (accuracy 48.8%, below the 50% coin-flip baseline) is reported as a *diagnostic experiment*, not a positive contribution claim. This result shows that direct weight transfer fails even under the simplified binary task, establishing a lower bound: the encoder has not learned a universal benign/attack boundary independent of dataset-specific value distributions. This negative result directly motivates our primary cross-dataset contribution: *schema-portable fine-tuned transfer*, which recovers 90.7% accuracy on TON-IoT via label-mapping and lightweight fine-tuning, without any feature re-engineering. We do not claim binary zero-shot viability; the experiment serves solely to separate taxonomy mismatch (multi-class failure) from representation failure (binary failure), both of which are confirmed, and both of which are resolved by fine-tuning.

Overall, the results support schema-portable *adaptation* as a practical cross-dataset pathway even when strict multi-class zero-shot transfer remains challenging.

5. Edge deployment and performance

5.1. Hardware platforms

All latency measurements are taken on an NVIDIA GB10 platform using batch size 1; Raspberry Pi 4 and Jetson Nano results are reported as FLOPs-based projections derived from publicly available FLOPs-per-watt specifications. Because a fixed τ yields the same exit distribution regardless of hardware, the speedup measured on the GB10 provides a first-order estimate for other platforms; projected values should be interpreted as order-of-magnitude estimates. Model size is dominated by the T5-small encoder (~ 35 M encoder parameters, out of the ~ 60 M full T5-small); the six exit heads add $< 2\%$ additional parameters.

Actual edge hardware benchmarks. To complement the FLOPs-based projections and address the gap between theoretical estimates and real-device behavior, we conducted actual inference benchmarks on a Raspberry Pi 5 Model B (aarch64, Cortex-A76, 4-core, Linux

Table 4
Conditions for schema-portable zero-shot transfer applicability.

Condition	Zero-shot applicable?	Reason
Same taxonomy + similar topology	✓	Class semantics aligned
Different label taxonomy	×	Class indices misaligned
Different schema, same taxonomy	Partial	Field names shift
Distribution shift (benign)	×	Spurious correlations

Table 5
Actual per-flow inference latency on real hardware (400 samples, early-exit $\tau = 0.1$, batch size 1).

Platform	Inference mode	Mean (ms)	p95 (ms)	n	Notes
<i>Measured (this work)</i>					
NVIDIA GB10	GPU FP32	0.72	–	400	Wall-clock
RPi5 Model B	CPU-only FP32	561	720	400	This work (actual)
RPi5 Model B	Hailo-8 INT8 NPU	2.21	2.22	2000	5-seed mean
<i>Projected (FLOPs-based estimate; not directly measured)</i>					
RPi4 Model B	CPU FP32	418	–	–	FLOPs estimate
Jetson Nano (4GB)	CPU FP32	12	–	–	FLOPs estimate

6.12.75)—the closest ARM device available in our laboratory to the originally projected RPi 4. Two configurations were measured, each with 400 samples and 5-sample warmup:

The CPU-only result (561 ms mean, 720 ms p95) demonstrates that FLOPs-based projections can underestimate actual latency by an order of magnitude: memory-bandwidth bottlenecks, tokenization overhead, and OS scheduling jitter are not captured by FLOPs counting alone. The Hailo-8 NPU result (2.21 ms mean) demonstrates that sub-10 ms real-time intrusion detection is achievable on sub-\$100 edge hardware, but only with a dedicated AI accelerator. The 254 $\times$ gap between CPU-only (561 ms) and NPU-assisted (2.21 ms) deployment demonstrates that *NPU acceleration is necessary, not optional*, for real-time IoT IDS.

5.2. Latency and throughput analysis

We decompose end-to-end latency on the GB10 into encoding and inference. At 256 tokens, tokenization takes 0.26 ms and embedding lookup adds 0.01 ms (17% of 1.64 ms total), while early-exit at $\tau = 0.1$ reduces end-to-end latency to 0.72 ms (mean exit at layer 1.17/6). All latencies reported in this section include field serialization and SentencePiece tokenization.

Considering the full pipeline, feature-based baselines incur a ~ 50 ms flow-aggregation step plus ~ 2 ms feature extraction in CIC-IDS-2017, yielding ~ 52 ms end-to-end even though the classifier itself runs in < 1 ms. By contrast, our raw-text early-exit pipeline runs in 0.72 ms on the GB10 (a first-order 72 $\times$ end-to-end reduction) by bypassing flow aggregation and feature extraction. Projected edge latencies are ~ 418 ms $\rightarrow$ 219 ms (RPi 4) and ~ 12 ms $\rightarrow$ 6.3 ms (Jetson Nano) for full versus early-exit inference (Fig. 4(a)). Actual measurements on RPi5 CPU-only yield 561 ms (early-exit $\tau = 0.1$), substantially higher than the FLOPs-based RPi4 projection (219 ms), reflecting tokenization and memory-bandwidth overhead not accounted for by FLOPs counting. RPi5 + Hailo-8 NPU reduces this to 2.21 ms, a 254 $\times$ reduction relative to CPU-only inference. These measurements reinforce that NPU-class hardware is required to achieve real-time (≤ 10 ms) per-flow IDS at the edge.

5.3. Pareto-efficient deployment

Fig. 3 plots the detection quality retention (relative to full-inference macro-F1) against inference speedup for each operating policy. All fixed- τ points retain $\geq 98.8\%$ of full-inference macro-F1 delivering up to 5.6 $\times$ speedup, with $\tau = 0.1$ near the knee of the frontier. The Pareto-optimized adaptive policy (Section 3.5) provides a complementary operating point at $\sim 3.5\times$ speedup with 98.3% retention.

We also estimate the normalized energy consumption per inference. Because early-exit reduces the number of active encoder layers under a

roughly constant power envelope, the energy savings mirror the latency reduction: approximately 80% across all four platforms. On Raspberry Pi 4, this corresponds to 2,820 mJ (full) versus ~ 552 mJ per inference at $\tau = 0.1$.

6. Security analysis

6.1. Payload-inaccessible traffic analysis

When traffic is encrypted, payload inspection is infeasible and a text-based IDS must rely on header metadata alone. To measure the impact, we construct a *header-only* variant by removing payload-related fields from the raw-text serialization (not by reintroducing engineered features), reducing the field set from 78 to 52 entries (33%). This setting mimics scenarios where payload content is unavailable (e.g., TLS/HTTPS), while header-level metadata remain observable. Header-only evaluation via payload-field removal is an established methodology: Anderson and McGrew (Anderson and McGrew, 2017) classify encrypted malware using only TLS handshake metadata; our experiment follows the same convention by retaining only NetFlow/IPFIX-accessible header and statistical fields. We note that Anderson and McGrew operate on real captured traffic, whereas our experiment simulates the payload-inaccessible condition via field removal from a labeled dataset; the two approaches share the same operative assumption (payload bytes are unavailable) but differ in implementation, and we make no claim of equivalence. We do not claim that this experiment fully reproduces all properties of a fully payload-inaccessible deployment: such environments can also hide, coarsen, or alter application-layer fields, and practical header-only analysis often depends on packet lengths, timing sequences, and flow dynamics. The result should therefore be interpreted as a controlled payload-removal stress test rather than as a complete payload-absent benchmark. As shown in Fig. 5, the header-only model achieves 97.2% accuracy, only 0.3 pp below the full-feature baseline (97.5%). The only notable degradation is benign (-3.15 pp), attributable to the loss of packet-size cues that help disambiguate benign from DDoS traffic. Overall, the results indicate that the model primarily relies on header-level behavior (timing, flags, packet counts) rather than payload content.

6.2. Adversarial robustness

Threat model: adversaries perturb the pre-processing stage by re-ordering or dropping fields, but do not modify packet payload content. Training-time poisoning attacks in federated settings are orthogonal and studied elsewhere (Nguyen et al., 2020).

Because raw-text representations encode network-flow fields as a serialized token sequence, the model is potentially sensitive to input permutations (Goodfellow et al., 2015). We evaluate two threat scenarios

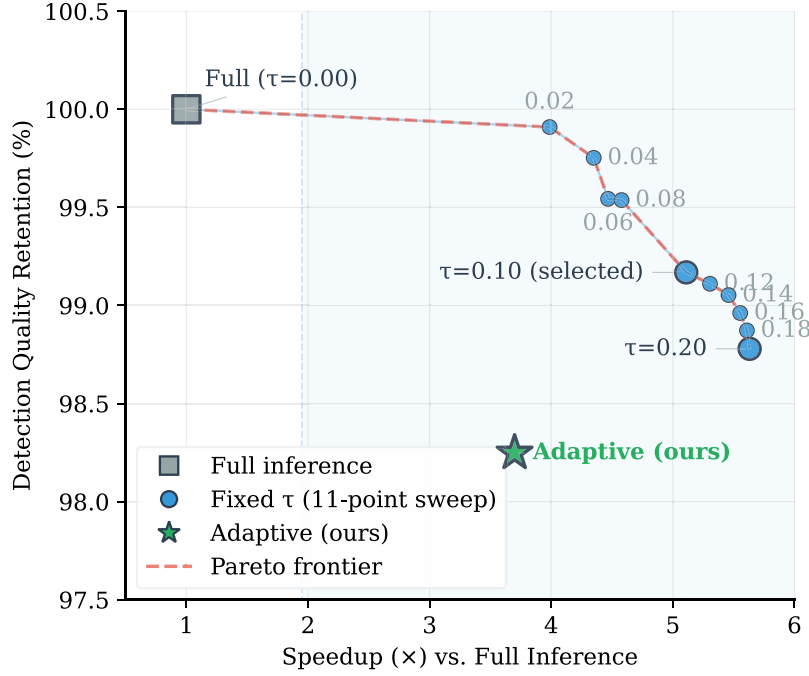


Fig. 3. Pareto trade-off between detection quality retention (% of full-inference macro-F1) and inference speedup across an 11-point threshold sweep ($\tau = 0.00, 0.02, \dots, 0.20$). Each circle corresponds to a fixed- τ policy evaluated without retraining; the star denotes the adaptive Pareto-optimized policy (Section 3.5).

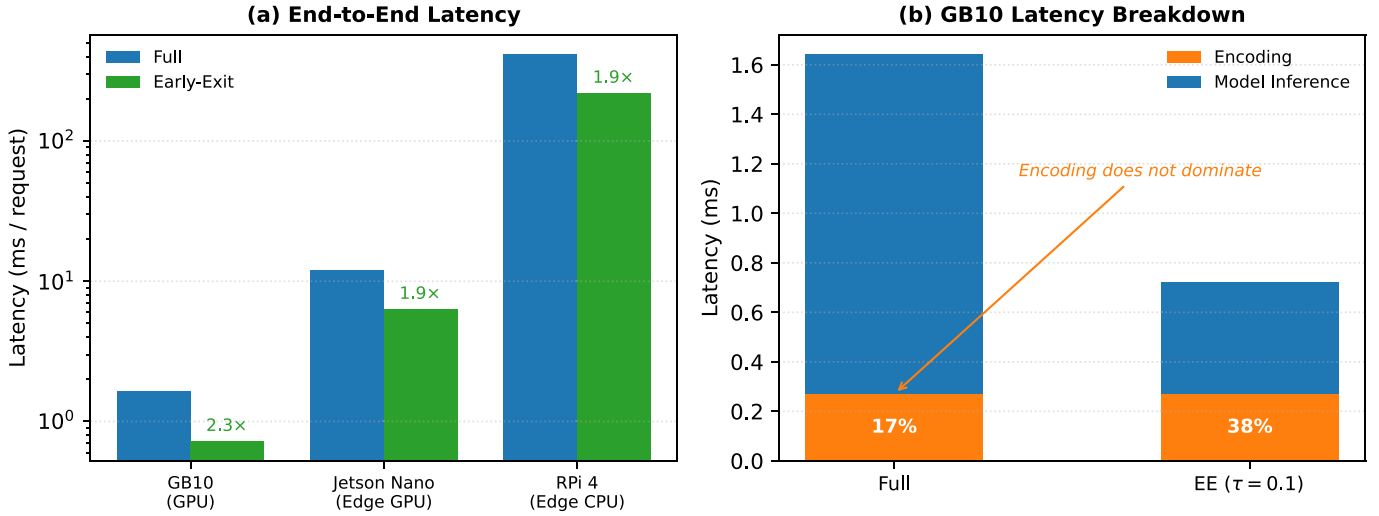


Fig. 4. Latency comparison between full and early-exit inference. (a) End-to-end latency (log scale) for GB10, Jetson Nano, and RPi 4; GB10 values are directly measured on our testbed, while edge-device values (RPi 4, Jetson Nano) are projected via device-specific scaling factors derived from each platform's published FLOP throughput. Green labels report the measured/projected end-to-end speedup; these wall-clock ratios are smaller than the $5.1\times$ FLOP/layer-reduction ratio because fixed tokenization overhead is not reduced by early-exit. Actual RPi 5 measurements (561 ms CPU-only; 2.21 ms with Hailo-8 NPU) are reported separately in Table 5. (b) GB10 latency breakdown into encoding and model inference: encoding is a fixed 0.27 ms (17% of full-depth, 38% under early-exit), confirming that text-encoding overhead does not dominate. Under the constant-power assumption ($E = P \times T$), the layer reduction translates to $\sim 80\%$ energy savings per inference.

(Fig. 6(a)): *feature reorder*, which randomly shuffles the field ordering (simulating schema drift), and *feature drop*, which randomly removes 10%–30% of fields (simulating missing or corrupted metadata). These perturbations evaluate a *preprocessing-level adversary*—one who disrupts field ordering or availability during feature extraction—which is practically motivated by the heterogeneous sensors and schema drift common in IoT deployments (e.g., varying CICFlowMeter versions across sites). They do *not* model a fully adaptive white-box adversary that crafts

minimal, protocol-valid perturbations tuned to the classifier's decision boundary. Such attacks require discrete token-space optimization, which is harder than continuous-space PGD and remains an open challenge for text-serialized network IDS. The results in this section should therefore be interpreted as preprocessing robustness guarantees, *not* as security guarantees against a model-aware evasion attacker. Under these perturbations the standard model drops from 97.7% clean accuracy to $\sim 35\%$, indicating that token order is a strong implicit signal.

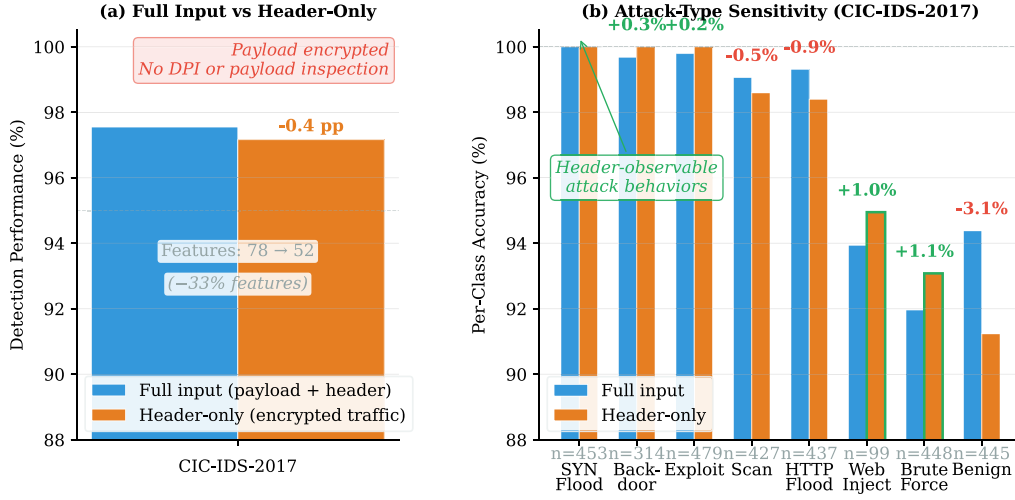


Fig. 5. Header-only (payload-inaccessible) evaluation on CIC-IDS-2017. Payload content is removed while retaining only raw header-level metadata (no engineered features). (a) Overall accuracy: removing all 26 payload-dependent features (78 $\rightarrow$ 52 fields, 33% reduction) degrades accuracy by only 0.3 pp. (b) Per-class comparison: most attack types are unaffected or even improve under the header-only setting. This is a controlled payload-removal stress test following established IDS methodology (Anderson and McGrew, 2017); effects of real TLS padding and traffic shaping are acknowledged as future work.

Table 6

Feature-value ℓ_∞ perturbation robustness on CICIOT2023 (CICIOT2023 model, $\tau = 0.30$, $n = 5322$). Values are MinMax-normalized; ϵ is relative to the $[0, 1]$ range.

ϵ	Accuracy (%)	Δ Acc (pp)	Macro-F1 (%)	Speedup
0.00 (clean)	89.3	–	87.5	2.22×
0.01	34.9	–54.4	32.1	1.09×
0.05	27.9	–61.4	24.8	1.06×
0.10	25.1	–64.2	20.0	1.06×

To mitigate this vulnerability we introduce a *hybrid robust* training strategy comprising three defenses: (i) *sorted features*, which alphabetically sort field names before serialization to remove order dependence; (ii) *feature-name embedding* (64-dimensional), which provides each field with a learnable identity independent of position; and (iii) *data augmentation* (shuffle probability 0.3, drop probability 0.1) during training (Madry et al., 2018). As Fig. 6(a) shows, the hybrid model recovers +52 pp under reorder (87.0% vs. 35.4%) and +58 pp under 30%-drop (91.3% vs. 33.5%), at a cost of only 3.9 pp in clean accuracy (93.8% vs. 97.7%). This provides a deployment option when schema drift or active pre-processing manipulation is suspected.

Feature-value perturbation sensitivity. To assess sensitivity beyond field ordering, we evaluate the CICIOT2023 model under a direct ℓ_∞ feature-value perturbation: uniform noise in $[-\epsilon, +\epsilon]$ is added to every numeric value in the serialized text (values are MinMax-normalized to $[0, 1]$), and the perturbed text is re-tokenized and passed through the model at $\tau = 0.30$. Table 6 summarizes the results. Even at $\epsilon = 0.01$ (1% of the normalized range), accuracy drops from 89.3% to 34.9% (–54.4 pp); at $\epsilon = 0.10$, accuracy falls to 25.1%. This brittleness is attributable to T5’s character-level number tokenization: small value changes (e.g., 0.122 $\rightarrow$ 0.131) alter the subword token sequence substantially, effectively producing out-of-distribution inputs for the encoder. These results demonstrate that text-serialized models face a distinct vulnerability relative to embedding-space attacks: even low-magnitude numerical perturbations at the representation layer can degrade classification reliability. Addressing this limitation requires more robust numerical encodings (e.g., fixed-point digit tokenization or dedicated numerical embedding modules) and is identified as a priority for future work.

6.3. Zero-day detection

To assess the model’s behavior when encountering previously unseen attack types, we perform a Leave-One-Attack-Out (LOAO) evaluation: for each class, a 7-class classifier is trained on the remaining categories and the held-out class is treated as unknown traffic. At inference, samples whose maximum softmax confidence falls below a threshold $\theta = 0.3$ are flagged as anomalous; this threshold is fixed a priori (not tuned on the test set) and applied uniformly across all LOAO runs. As shown in Fig. 6(b), known-class accuracy remains high (avg. 97.6%) across held-out scenarios.

The entropy-based detection rate varies widely across attack types. Categories whose behavioral profiles diverge substantially from all training classes—benign (44.9%), HTTP flood (33.1%), and scanning (22.1%)—produce elevated entropy and are flagged at moderate rates. Conversely, attacks that closely resemble a known category (e.g., SYN flood mimicking other DDoS patterns) yield low entropy and largely evade detection ($\leq 1.3\%$). This behavior stems from the model’s confidence being calibrated to the training distribution: an unseen attack whose traffic patterns overlap with a known class is assigned high confidence to that class, suppressing exit-layer entropy and preventing flagging. We therefore frame the signal not as a standalone zero-day classifier but as an *unknown-traffic awareness* mechanism: uncertain flows can be escalated to full-depth analysis or SOC review (Section 5.3). More sophisticated uncertainty quantification methods (e.g., Monte Carlo Dropout, deep ensembles) may improve detection of behaviorally similar unknowns and are left for future work (Section 7.4).

7. Discussion

7.1. Schema-portable vs. feature-based trade-offs

The best feature-engineered baseline (Gradient Boosting, 98.5%) outperforms our early-exit model (97.0% at $\tau = 0.1$) by 1.5 percentage points (Table 3). This gap, however, obscures three hidden costs of the feature-based paradigm. First, the 78-feature pipeline depends on CICIOT2023-style flow aggregation, which is undefined for encrypted or non-standard IoT protocols; our header-only experiment (Section 6.1) shows that a raw-text encoder retains 97.2% accuracy even when payload inspection is unavailable. Second, hand-crafted features are dataset-specific: the same Gradient Boosting model cannot

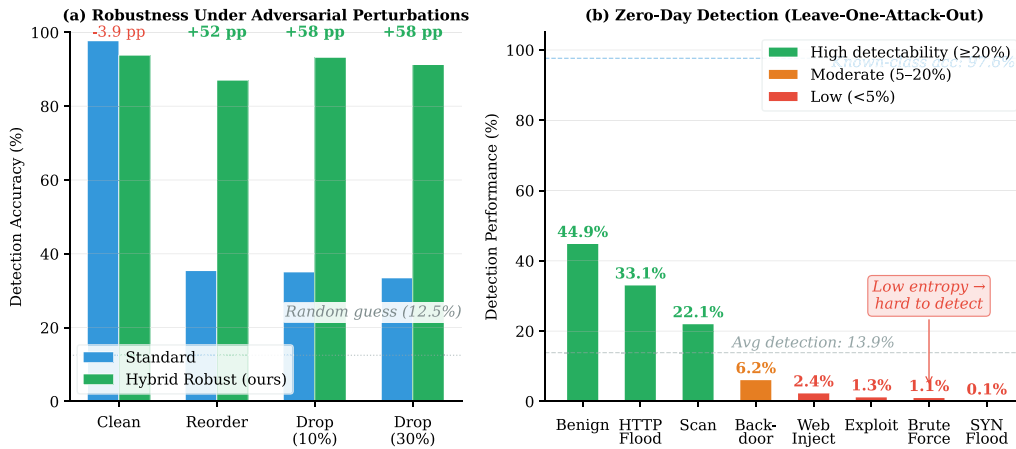


Fig. 6. Robustness and zero-day evaluation. (a) Accuracy under feature reordering and dropping, showing the hybrid robust training recovers +52 pp/+58 pp with a 3.9 pp clean-accuracy penalty. (b) LOAO zero-day detection using confidence threshold $\theta = 0.3$.

be transferred to TON-IoT or UNSW-NB15 without re-engineering the feature set, whereas fine-tuning our text encoder reaches 90.7% on TON-IoT (Section 4.6). Third, flow aggregation itself introduces a 50 ms latency floor (Section 5.2), negating any latency advantage of a low-overhead classifier; our end-to-end pipeline operates in 0.72 ms (72× reduction).

On CICIOT2023, the accuracy gap narrows to only 1.8 pp (Gradient Boosting 91.1% vs. ours 89.3%), demonstrating that the pipeline generalizes effectively even to datasets with 34 fine-grained attack types compressed to 8 classes; the feature-based model additionally requires re-engineering its 46-feature pipeline for each new dataset. In sum, the accuracy cost (1.5 pp on CIC-IDS-2017, 1.8 pp on CICIOT2023) purchases deployment flexibility that cannot be achieved by optimizing within the feature-engineering paradigm. We note that recent Transformer-based IDS methods such as ET-BERT (Lin et al., 2022) operate on raw byte sequences or TLS metadata, making direct accuracy comparison infeasible due to input-modality differences. Our full-inference baseline ($\tau = 0$ in Table 2) effectively serves as the *Transformer-without-early-exit* reference; the contribution of this work is the adaptive exit mechanism and schema-portable representation atop the shared backbone, not the backbone architecture itself.

7.2. Early-exit as security architecture

Conventional uses of early-exit focus on latency reduction, treating confidence thresholds as a knob for trading accuracy against speed. Our experiments point to a broader role: early-exit can function as a *resource-aware security decision mechanism*. Three observations support this reframing. (i) The well-calibrated confidence scores (ECE = 0.0125, Table 2) ensure that the exit decision reflects genuine model certainty rather than arbitrary logit magnitude, making it a reliable basis for triage. (ii) The dual-track deployment strategy (Section 5.3)—where edge gateways apply an aggressive threshold for rapid triage while a centralized SOC retains full-depth inference for the ~2% of ambiguous flows—mirrors the tiered alert architecture already standard in network security operations. (iii) The LOAO evaluation (Section 6.3) shows that elevated exit-layer entropy serves as an implicit anomaly signal, enabling *unknown-traffic awareness* without a dedicated out-of-distribution detector.

This architectural view also motivates dynamic threshold management. Under a resource-budget and protocol-flooding stress test (hardware-profile dependent), $\tau = 0.15$ balances accuracy (94.8%) and resource usage on a 1 GB-class edge node; on a 512 MB device the optimal threshold rises to $\tau = 0.35$ (88.9%) to preserve processing headroom. When protocol-level flooding is detected—e.g., MQTT

spam degrades accuracy by 12 pp and mixed flooding by 16 pp—the guard policy forces full-depth inference (exit-rate gap $> 18\%$ or accuracy gap $> 15\%$), effectively trading latency for security until the attack subsides. The early-exit threshold thus becomes not merely an inference-time parameter but an *observable, controllable security knob* that adapts to both hardware constraints and threat conditions. Critically, full-depth inference remains the *security floor*: any sample that fails to reach sufficient confidence at intermediate layers is guaranteed to receive the complete six-layer analysis, ensuring that the early-exit mechanism never silently degrades detection assurance. A natural extension is *class-conditional* exit policies: because false negatives (missed attacks) carry far higher cost than false positives in an IDS context, the threshold could be tightened when the current head predicts *benign*—requiring deeper confirmation before releasing a non-alert verdict—while permitting earlier exit for high-confidence attack predictions that trigger immediate escalation. Our training objective already encodes this asymmetry through the false-negative penalty ($\alpha_{fn} = 2.0$, Section 3.7); class-conditional thresholds would provide the corresponding inference-side mechanism. We leave this extension to future work, as it requires per-class threshold tuning on a deployment-specific cost matrix.

7.3. Comparison with representative related methods

Table 1 (Section 2) already summarizes capability gaps across the literature; Table 7 below complements that analysis by addressing *performance comparability*: whether accuracy figures can be compared directly and, if not, why. Methods evaluated on CIC-IDS-2017 with the same 8-class taxonomy use the same held-out test split as Table 3 and are directly comparable; all others use different datasets, input modalities, or task formulations that preclude fair numerical comparison.

Among directly comparable methods on CIC-IDS-2017, our full-inference model (97.7%) closes to within 0.8 pp of Gradient Boosting (98.5%) without any hand-crafted features, and the early-exit variant (97.0%) is within 0.5 pp of MLP (97.5%) while eliminating the 50 ms aggregation overhead (detailed in Section 7.1). Cross-dataset results on CICIOT2023 are discussed separately in Section 4.4 and Section 7.1. Methods in the “Indirect” rows use different datasets, input modalities, or task formulations; reporting their accuracy figures alongside ours would be misleading and is therefore omitted. We include them to delimit the scope of this work: FedIDS and MoARNN-AM address federated or UAV-specific deployments; DeepPacket and ET-BERT operate on raw bytes or TLS metadata; BranchyNet provides the early-exit concept but not an IDS solution; EvoAAE targets unsupervised anomaly detection rather than multi-class supervised classification.

Table 7

Performance comparison with representative related methods, grouped by comparability. Direct entries share the same dataset and held-out split as Table 3; CICIOT2023 cross-dataset results are discussed in Section 4.4 and Section 7.1. For capability gaps (schema portability, early-exit, NPU validation) see Table 1 in Section 2.

Method	Venue	Dataset	Acc. (%)	Comparability
<i>CIC-IDS-2017 — same 8-class, 3102-sample split as Table 3</i>				
CICFlowMeter + GB (Sharafaldin et al., 2018)	ICISSP 2018	CIC-IDS-2017	98.5	Direct
CICFlowMeter + RF (Sharafaldin et al., 2018)	ICISSP 2018	CIC-IDS-2017	98.0	Direct
MLP	—	CIC-IDS-2017	97.5	Direct
LSTM	—	CIC-IDS-2017	97.3	Direct
This work (full)	—	CIC-IDS-2017	97.7	Direct
This work (EE, $\tau=0.1$)	—	CIC-IDS-2017	97.0	Direct
<i>Other literature — indirect comparison only</i>				
FedIDS (Bhavsar et al., 2024)	IEEE Access 2024	Transportation IoT	—	Indirect: different domain
DeepPacket (Lotfollahi et al., 2020)	Soft Comput. 2020	ISCX-VPN	—	Indirect: different dataset & modality
ET-BERT (Lin et al., 2022)	WWW 2022	CSTNET-TLS	—	Indirect: different dataset & modality
BranchyNet (Teerapittayanon et al., 2016)	ICPR 2016	MNIST	—	Indirect: computer-vision task
EvoAAE (Zeng et al., 2025)	IEEE TR 2025	IIoT	—	Indirect: anomaly detection
MoARNN-AM (Lu et al., 2025)	IEEE TCE 2025	UAV	—	Indirect: different domain

7.4. Limitations and future work

Several limitations warrant transparency. First, cross-dataset transfer remains uneven: fine-tuning on TON-IoT achieves 90.7% accuracy, but UNSW-NB15 plateaus at 67% (Section 4.6), primarily because UNSW-NB15 uses a fundamentally different traffic generation methodology and attack labeling taxonomy than CIC-IDS-2017—a domain gap that raw-text encoding alone cannot bridge.

Scope of zero-shot transfer. Zero-shot multi-class transfer is limited by attack taxonomy misalignment across benchmarks. When training and test datasets share the same label definitions, the schema-portable representation enables direct weight transfer; when they differ (as in CIC-IDS-2017 → TON-IoT), label re-mapping and fine-tuning are required. Feature-based models face the same taxonomy problem *plus* the additional requirement of rebuilding the extraction pipeline; schema-portability addresses only the latter. We therefore position this contribution as eliminating *engineering effort* in cross-dataset adaptation rather than enabling zero-shot multi-class generalization.

Second, zero-day detection via exit-layer entropy is effective for behaviorally distinctive attacks (HTTP flood 33.1%, scanning 22.1%) but nearly ineffective when the unknown attack closely resembles a known class (SYN flood $\leq 1.3\%$; Section 6.3); the entropy signal is therefore best understood as an *anomaly indicator*, not a complete out-of-distribution detector. Third, the hybrid robust training that recovers +52pp under field-order perturbation incurs a 3.9pp clean-accuracy penalty (Section 6.2), making it best suited for high-threat environments rather than as a universal default. The current robustness experiment covers random field reordering and dropping only (preprocessing-level adversary) and does not constitute a security guarantee against a fully adaptive, model-aware attacker. Four concrete future directions are identified: (i) *gradient-guided token substitution*—searching for minimal discrete perturbations over flow field values that shift the model’s prediction while preserving semantic validity; (ii) *protocol-valid evasion*—crafting network traffic that maintains legitimate handshake behavior (e.g., valid TCP flags, realistic port numbers) while displacing classification features across the decision boundary; and (iii) *adaptive entropy attacks*—adversarially steering samples toward high-entropy exit distributions to reduce detection confidence or force late-layer exits that increase latency; and (iv) *robust numerical tokenization*—replacing character-level number encoding with fixed-point digit representations or dedicated numerical embedding modules to eliminate the high sensitivity to feature-value perturbations documented in Table 6. These directions require discrete optimization under protocol constraints and remain an open research challenge for text-serialized IDS. On CICIOT2023, four of eight classes fall below 70% F1 (brute force 55.8%, scanning 62.2%, exploit 66.7%, web injection 57.7%), primarily because the 34→8 label mapping groups semantically diverse attacks into coarse categories (e.g., DNS spoofing and uploading attacks both

map to `iot_exploit`). On CIC-IDS-2017, the `web_injection` class remains the weakest category ($F1 = 0.795$), limited by sample size and behavioral overlap with brute-force traffic. On the deployment side, the newly added RPi5 CPU-only and RPi5+Hailo-8 measurements provide real-device latency evidence, but the RPi4 and Jetson Nano values remain FLOPs-based projections rather than sustained measurements on those exact platforms; actual performance will vary with inference framework (e.g., ONNX Runtime, TensorFlow Lite), weight quantization (INT8/FP16), OS scheduling, and thermal throttling. Long-duration evaluation under realistic network load remains necessary before production deployment.

CPU-only edge deployment. Actual benchmarks on RPi5 (CPU-only FP32) reveal 561 ms mean per-flow latency, confirming that ARM CPU inference is prohibitively slow for real-time IoT IDS without hardware acceleration. A dedicated NPU (Hailo-8: 2.21 ms, 254× faster) is necessary for sub-10 ms deployment. This is a more stringent constraint than the FLOPs-based projections implied, and it shifts the practical deployment boundary: edge IDS based on Transformer encoders requires NPU support. Future work should evaluate NPU-portable quantization formats (e.g., ONNX, TFLITE) and alternative accelerator platforms (Jetson Orin, Coral TPU) to characterize the cost-performance frontier more completely.

These limitations suggest several promising directions. *Uncertainty quantification* via Monte Carlo Dropout or deep ensembles could improve zero-day detection for behaviorally similar unknowns, complementing the entropy-based signal. *Integrated adversarial training* that jointly optimizes the early-exit objective and robustness to input perturbation may reduce the clean-accuracy cost of the current two-stage approach. *Streaming inference*—classifying individual packets rather than aggregated flows—would eliminate the residual encoding overhead and enable sub-millisecond detection. *Multi-modal fusion* combining packet payload, flow statistics, and host-level logs could strengthen recognition of underrepresented attack types such as web injection, where a single modality provides insufficient discriminative signal.

8. Conclusion

This paper presented an intrusion detection framework for IoT networks that bypasses handcrafted feature selection. A T5-small encoder classifies raw-text traffic representations across eight categories at 97.7% accuracy, and an adaptive early-exit mechanism — whose threshold is selected via Pareto optimization — delivers a 5.11× speedup at the cost of only 0.7pp in accuracy. Exit-layer confidence is well calibrated ($ECE = 0.0125$), supporting reliable triage at the network edge. Because the model operates on serialized header fields rather than hand-crafted flow statistics, it generalizes to unseen datasets (90.7% on TON-IoT) and remains effective under payload-inaccessible (header-only) conditions (97.2% accuracy, $-0.3pp$). Independent evaluation

Table A.8

Device parameters used in energy estimation. TDP values are manufacturer-specified upper bounds.

Device	Inference mode	TDP (W)	Full-depth t (ms)
NVIDIA GB10	GPU FP32	65	1.64
Raspberry Pi 4	CPU FP32 (proj.)	7.6	418
Jetson Nano	CPU FP32 (proj.)	10	12
RPi5 + Hailo-8	NPU INT8 (actual)	10	2.21

on CICIOT2023 (34 attack types, 46 native features) achieves 89.3% accuracy with 2.22 $\times$ speedup, confirming that the pipeline generalizes across heterogeneous IoT datasets without feature re-engineering. A hybrid robust training strategy recovers +52pp accuracy under field-order perturbations, and entropy-based anomaly scoring provides unknown-traffic awareness for zero-day escalation. Actual hardware benchmarks on Raspberry Pi 5 show that CPU-only inference requires 561 ms per flow, while RPi5 + Hailo-8 NPU achieves 2.21 ms (254 $\times$ speedup), indicating that dedicated NPU acceleration is likely required for real-time edge deployment. GPU inference on the NVIDIA GB10 achieves 0.72 ms early-exit latency, 72 $\times$ faster than conventional feature-extraction pipelines, with $\sim$ 80% energy savings per inference.

The upshot is that early-exit inference can serve as more than an acceleration technique—it enables a tiered deployment in which edge gateways handle rapid triage while a centralized SOC retains full-depth analysis for the marginal fraction of uncertain flows.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) in order to improve language and readability of the manuscript. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Energy estimation methodology

Energy per inference is estimated as $E = P_{\text{active}} \times t_{\text{infer}}$, where P_{active} is the device TDP (thermal design power) under sustained inference load and t_{infer} is the per-flow latency. Table A.8 lists the device parameters used. Because early-exit reduces t_{infer} by a factor equal to the layer-reduction ratio (e.g., 5.11 $\times$ speedup yields a 5.11 $\times$ energy reduction under constant power), the $\sim$ 80% energy saving is a derived quantity, not an independently measured one. Actual energy consumption will differ based on dynamic frequency scaling, DRAM refresh, and OS background load.

Appendix B. Tokenizer length sensitivity

Table B.9 reports accuracy and macro-F1 across three maximum sequence lengths $L \in \{128, 256, 512\}$ on CIC-IDS-2017. $L = 256$ is used throughout the paper as the operating point that maximizes performance without incurring the memory overhead of longer contexts; 70% of flows exceed 128 tokens when all 78 fields are serialized.

$L = 512$ yields no accuracy gain (+0.1 pp macro-F1) while nearly doubling peak memory, confirming $L = 256$ as the Pareto-optimal operating point.

Table B.9

Accuracy and macro-F1 vs. tokenizer maximum length L on CIC-IDS-2017 (seed 42, full-depth inference).

Max length L	Accuracy (%)	Macro-F1 (%)	Peak mem (MB)
128	81.2	78.4	$\sim$ 14
256	97.7	96.2	$\sim$ 28
512	97.7	96.3	$\sim$ 52

Table C.10

Per-class F1 on CIC-IDS-2017 test set (seed 42). n : test-set samples per class.

Class	n	Full-depth F1	Early-exit F1 ($\tau = 0.1$)
Benign	445	0.980	0.959 (−2.1 pp)
DDoS HTTP Flood	437	0.981	0.979 (−0.2 pp)
DDoS SYN Flood	453	0.994	0.993 (−0.1 pp)
IoT Backdoor	314	0.986	0.984 (−0.2 pp)
IoT Brute Force	448	0.978	0.977 (−0.1 pp)
IoT Exploit	479	0.972	0.972 ($\pm$ 0.0 pp)
IoT Scanning	427	0.974	0.973 (−0.1 pp)
Web Injection	99	0.795	0.778 (−1.7 pp)
Macro avg.	3102	0.962	0.954 (−0.8 pp)

Appendix C. Per-class F1 under full inference and early-exit

Table C.10 reports full per-class F1 for all eight attack categories under both full-depth inference and early-exit ($\tau = 0.1$) on CIC-IDS-2017. Aggregate metrics in the main paper (Table 2) reflect these per-class values; this table is provided for complete reproducibility.

Web injection is the most affected class (−1.7 pp), consistent with its limited sample size ($n=99$) and behavioral overlap with brute-force traffic noted in Section 7.1. All other classes degrade by ≤ 0.2 pp, confirming that early-exit precision loss is concentrated on the statistically weakest category.

Data availability

The datasets used in this study (CIC-IDS-2017, CICIOT2023, TON-IoT, UNSW-NB15) are publicly available from their respective sources. The code and pre-computed results are available at <https://github.com/DevSecOpsLab-CSIE-NPU/schema-portable-ee-ids>.

References

- Al-Shurbaji, T., Anbar, M., Manickam, S., Hasbullah, I., Alfricate, N., Alabsi, B.A., Alzighaibi, A., Hashim, H., 2025. Deep learning-based intrusion detection system for detecting IoT botnet attacks: A review. *IEEE Access* <http://dx.doi.org/10.1109/ACCESS.2025.3526711>.
- Alatawi, M.N., 2025. EdgeGuard-IoT: 6G-enabled edge intelligence for secure federated learning. *CMC* <http://dx.doi.org/10.32604/cmc.2025.066606>.
- Alenazi, M.M., Alassafi, A., Alshamrani, A., 2025. EdgeFL-crypto: Federated split learning architecture for IoT-based cryptocurrency volatility prediction. <http://dx.doi.org/10.21203/rs.3.rs-7873990/v1>, Preprint.
- Anderson, B., McGrew, D., 2017. Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity. In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD*, pp. 1723–1732. <http://dx.doi.org/10.1145/3097983.3098163>.
- Anoh, N.G., Kone, T., Adépo, J., M'MOH, J.F., Babri, M., 2024. IoT intrusion detection system based on machine learning algorithms using the UNSW-NB15 dataset. *Int. J. Adv. Sci. Res. Eng.* <http://dx.doi.org/10.31695/ijasre.2024.1.3>.
- Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K., 2022. Dos and don'ts of machine learning in computer security. In: *USENIX Security Symposium*, pp. 3971–3988.
- Bakhtiarnia, A., Zhang, Q., Iosifidis, A., 2021. Multi-exit vision transformer for dynamic inference. <http://dx.doi.org/10.48550/arXiv.2106.15183>, arXiv. URL <https://arxiv.org/abs/2106.15183>.
- Bhavsar, M., Bekele, Y., Roy, K., Kelly, J., Limbrick, D.B., 2024. FL-IDS: Federated learning-based intrusion detection system using edge devices for transportation IoT. *IEEE Access* <http://dx.doi.org/10.1109/ACCESS.2024.3386631>.
- Chai, Y., Chen, X., Qiu, J., Du, L., Xiao, Y., Feng, Q., Ji, S., Tian, Z., 2025. MaIFSCIL: A few-shot class-incremental learning approach for malware detection. *IEEE Trans. Inf. Forensics Secur.* 20, 2999–3014. <http://dx.doi.org/10.1109/TIFS.2024.3516565>.

- Chai, Y., Du, L., Qiu, J., Yin, L., Tian, Z., 2023. Dynamic prototype network based on sample adaptation for few-shot malware detection. *IEEE Trans. Knowl. Data Eng.* 35 (5), 4754–4766. <http://dx.doi.org/10.1109/TKDE.2022.3142820>.
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*. NAACL, pp. 4171–4186. <http://dx.doi.org/10.18653/v1/N19-1423>.
- Goodfellow, I.J., Shlens, J., Szegedy, C., 2015. Explaining and harnessing adversarial examples. In: *International Conference on Learning Representations*. ICLR, <http://dx.doi.org/10.48550/arXiv.1412.6572>.
- Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks. In: *Proceedings of the 34th International Conference on Machine Learning*. ICML, In: *Proceedings of Machine Learning Research*, vol. 70, PMLR, pp. 1321–1330.
- Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Jagatheesaperumal, S.K., Rahouti, M., Alfatem, A., Ghani, N., Quy, V.K., Chehri, A., 2024. Enabling trustworthy federated learning in industrial IoT. *IEEE IOTM* <http://dx.doi.org/10.1109/IOTM.001.2300274>.
- Jia, Y., Gu, Z., Du, L., Long, Y., Wang, Y., Li, J., Zhang, Y., 2023. Artificial intelligence enabled cyber security defense for smart cities: A novel attack detection framework based on the MDATA model. *Knowl.-Based Syst.* 276, 110781. <http://dx.doi.org/10.1016/j.knsys.2023.110781>.
- Kairouz, P., McMahan, H.B., et al., 2021. Advances and open problems in federated learning. *Found. Trends ML* <http://dx.doi.org/10.1561/22000000083>.
- Karimireddy, S.P., Kale, S., Mohri, M., Reddi, S.J., Stich, S.U., Suresh, A.T., 2020. SCAFFOLD: Stochastic controlled averaging for federated learning. *ICML* <http://dx.doi.org/10.48550/arXiv.1910.06378>.
- Kaya, Y., Hong, S., Dumitras, T., 2019. Shallow-deep networks: Understanding and mitigating network overthinking. In: *International Conference on Machine Learning*. ICML, pp. 3301–3310. <http://dx.doi.org/10.48550/arXiv.1810.07052>.
- Koroniatis, N., Moustafa, N., Sitnikova, E., Turnbull, B., 2019. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. *Future Gener. Comput. Syst.* 100, 779–796. <http://dx.doi.org/10.1016/j.future.2019.05.041>.
- Krishna, E., Arunkumar, T., 2021. Hybrid particle swarm and gray wolf optimization algorithm for IoT intrusion detection system. *Int. J. Intell. Eng. Syst.* <http://dx.doi.org/10.22266/ijies2021.0831.07>.
- Li, J., Othman, M.S., Chen, H., Yusuf, L.M., 2024. Optimizing IoT intrusion detection system: feature selection versus feature extraction in machine learning. *J. Big Data* <http://dx.doi.org/10.1186/s40537-024-00892-y>.
- Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., Yu, J., 2022. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In: *Proceedings of the ACM Web Conference*. WWW, pp. 633–642.
- Liu, W., Zhou, P., Zhao, Z., Wang, Z., Deng, H., Ju, Q., 2020. FastBERT: a self-distilling BERT with adaptive inference time. *ACL* <http://dx.doi.org/10.18653/v1/2020.acl-main.99>.
- Lotfollahi, M., Siavoshani, M.J., Zade, R.S.H., Saberian, M., 2020. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Comput.* 24 (3), 1999–2012. <http://dx.doi.org/10.1007/s00500-019-04030-2>.
- Lu, K.-D., Zhang, B.-X., Xu, Y., Shen, Y., Wu, Z.-G., 2025. MoARNN-AM: Multi-objective automated recurrent neural network with attention mechanism for cyber-attack detection of UAV. *IEEE Trans. Consum. Electron.* <http://dx.doi.org/10.1109/TCE.2025.3642083>, Early Access.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A., 2018. Towards deep learning models resistant to adversarial attacks. In: *International Conference on Learning Representations*. ICLR, <http://dx.doi.org/10.48550/arXiv.1706.06083>.
- McNemar, Q., 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12 (2), 153–157. <http://dx.doi.org/10.1007/BF02295996>.
- Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., Elovici, Y., 2018. N-BalIoT: Network-based detection of IoT botnet attacks. *IEEE Pervasive Comput.* <http://dx.doi.org/10.1109/MPRV.2018.03367731>.
- Moustafa, N., 2021. A new distributed architecture for evaluating AI-based security systems at the edge: Network TONIoT datasets. *Sustain. Cities Soc.* 72, 102994. <http://dx.doi.org/10.1016/j.scs.2021.102994>.
- Moustafa, N., Slay, J., 2015. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In: *Military Communications and Information Systems Conference*. MilCIS, pp. 1–6. <http://dx.doi.org/10.1109/MilCIS.2015.7348942>.
- Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A., 2023. CIC-IoT-2023: A comprehensive dataset for large-scale security analysis. *IEEE Access* <http://dx.doi.org/10.1109/ACCESS.2023.3298815>.
- Nguyen, T.D., Rieger, P., Miettinen, M., Sadeghi, A., 2020. Poisoning attacks on federated learning-based IoT intrusion detection system. In: *Proceedings 2020 Workshop on Decentralized IoT Systems and Security*. <http://dx.doi.org/10.14722/diss.2020.23003>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* 21 (140), 1–67. <http://dx.doi.org/10.5555/3455716.3455856>.
- Ren, Y., Xiao, Y., Zhou, Y., Zhang, Z., Tian, Z., 2022. CSKG4APT: A cybersecurity knowledge graph for advanced persistent threat organization attribution. *IEEE Trans. Knowl. Data Eng.* <http://dx.doi.org/10.1109/TKDE.2022.3175719>.
- Sayed, N., Shoaib, M., Ahmed, W., Qasem, S.N., Albarrak, A.M., Saeed, F., 2022. Augmenting IoT intrusion detection system performance using deep neural network. *Comput. Mater. Contin.* <http://dx.doi.org/10.32604/cmc.2023.030831>.
- Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *International Conference on Information Systems Security and Privacy*. ICISPP, pp. 108–116. <http://dx.doi.org/10.5220/0006639801080116>.
- Sommer, R., Paxson, V., 2010. Outside the closed world: On using machine learning for network intrusion detection. In: *IEEE Symposium on Security and Privacy (S&P)*. pp. 305–316. <http://dx.doi.org/10.1109/SP.2010.25>.
- Tan, J., Zheng, T., Jin, H., Liu, Y., Zhang, H., Sun, Y.-B., 2025. A strategy-making method for PLoT PLC honeypot defense against attacks based on the time-delay evolutionary game. *IEEE Trans. Inf. Forensics Secur.* 20, 11528–11543. <http://dx.doi.org/10.1109/TIFS.2025.3601442>.
- Tang, P., Zhu, P., Li, T., Appalaraju, S., Mahadevan, V., Manmatha, R., 2024. DEED: Dynamic early exit on decoder for accelerating encoder-decoder transformer models. *NAACL* <http://dx.doi.org/10.18653/v1/2024.findings-naacl.9>.
- Teerapittayanon, S., McDanel, B., Kung, H.T., 2016. BranchyNet: Fast inference via early exiting from deep neural networks. In: *International Conference on Pattern Recognition*. ICPR, pp. 2464–2469. <http://dx.doi.org/10.1109/ICPR.2016.7900006>.
- Turcotte, M.J.M., Kent, A.D., Hash, C., 2018. Unified host and network data set. *Data Sci. Cyber-Security* 1–22. http://dx.doi.org/10.1142/9781786345646_0001.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, <http://dx.doi.org/10.48550/arXiv.1706.03762>.
- Wang, J., Chen, K., Chen, G., Shou, L., McAuley, J., 2022. SkipBERT: Efficient inference with shallow layer skipping. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 7287–7301. <http://dx.doi.org/10.18653/v1/2022.acl-long.503>, URL <https://aclanthology.org/2022.acl-long.503/>.
- Wang, Z., Zhou, Y., Liu, H., Qiu, J., Fang, B., Tian, Z., 2024. ThreatInsight: Innovating early threat detection through threat-intelligence-driven analysis and attribution. *IEEE Trans. Knowl. Data Eng.* 36 (12), 9388–9402. <http://dx.doi.org/10.1109/TKDE.2024.3474792>.
- Xin, J., Tang, R., Lee, J., Yu, Y., Lin, J., 2020. DeeBERT: Dynamic early exit for BERT. *ACL* <http://dx.doi.org/10.18653/v1/2020.acl-main.204>.
- Xin, J., Tang, R., Yu, Y., Lin, J., 2021. BERTxIT: Early exiting for BERT with better fine-tuning and extension to regression. In: *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Association for Computational Linguistics, pp. 91–104. <http://dx.doi.org/10.18653/v1/2021.eacl-main.8>, URL <https://aclanthology.org/2021.eacl-main.8/>.
- Xu, G., Hao, J., Shen, L., Hu, H., Luo, Y., Lin, H., Shen, J., 2023. LGViT: Dynamic early exiting for accelerating vision transformer. <http://dx.doi.org/10.48550/arXiv.2308.00255>, arXiv. URL <https://arxiv.org/abs/2308.00255>.
- Ye, Y., Li, S., Liu, F., Tang, Y., Hu, W., 2020. EdgeFed: Optimized federated learning based on edge computing. *IEEE Access* <http://dx.doi.org/10.1109/ACCESS.2020.3038287>.
- Ye, D., Lin, Y., Huang, Y., Sun, M., 2021. TR-BERT: Dynamic token reduction for accelerating BERT inference. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, pp. 5798–5809. <http://dx.doi.org/10.18653/v1/2021.naacl-main.463>, URL <https://aclanthology.org/2021.naacl-main.463/>.
- Zeng, G.-Q., Yang, Y.-W., Lu, K.-D., Geng, G.-G., Weng, J., 2025. Evolutionary adversarial autoencoder for unsupervised anomaly detection of industrial internet of things. *IEEE Trans. Reliab.* 74 (3), 3454–3468. <http://dx.doi.org/10.1109/TR.2025.3528256>.
- Zhang, H., 2025. Development of an intelligent intrusion detection system for IoT networks using deep learning. *Discov. Internet Things* <http://dx.doi.org/10.1007/s43926-025-00177-7>.
- Zhao, Y., Li, M., Lai, L., Suda, N., Civin, D., Chandra, V., 2018. Federated learning with non-IID data. <http://dx.doi.org/10.48550/arXiv.1806.00582>, arXiv.
- Zhou, Y., Ren, Y., Yi, M., Xiao, Y., Tan, Z., Moustafa, N., Tian, Z., 2023. CDTier: A Chinese dataset of threat intelligence entity relationships. *IEEE Trans. Sustain. Comput.* 8 (4), 627–638. <http://dx.doi.org/10.1109/TSUSC.2023.3240411>.
- Zhou, Y., Wang, Z., Jiang, Y., Ma, B., Wang, R., Liu, Y., Zhao, Y., Tian, Z., 2025. AEKG4APT: An AI-enhanced knowledge graph for advanced persistent threats with large language model analysis. *ACM Trans. Intell. Syst. Technol.* <http://dx.doi.org/10.1145/3735645>.
- Zhou, W., Xu, C., Ge, T., McAuley, J., Xu, K., Wei, F., 2020. BERT loses patience: Fast and robust inference with early exit. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, <http://dx.doi.org/10.48550/arXiv.2006.04152>.